

AIM

To write a program to implement error detection and correction using the **Hamming Code** concept and verify the error correction feature.

ALGORITHM

Sender

1. Read the input text.
2. Convert each character into its 8-bit binary representation.
3. Divide the binary data into 4-bit groups.
4. Add redundant parity bits using Hamming (7,4) code.
5. Store the encoded data in the `channel.txt` file.

Receiver

1. Read the encoded data from `channel.txt`.
2. Divide the data into 7-bit Hamming codewords.
3. Calculate the parity bits.
4. Find the error position using the syndrome.
5. Correct the erroneous bit.
6. Remove the redundant parity bits.
7. Convert the remaining binary data back to ASCII text.
8. Display the original message.

PROGRAM

Sender Program

Python

```
def hamming_encode(data):  
    result = ""  
  
    for i in range(0, len(data), 4):  
        d = data[i:i+4]  
  
        while len(d) < 4:  
            d += "0"
```

```

d1, d2, d3, d4 = map(int, d)

# Hamming (7,4)
p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p4 = d2 ^ d3 ^ d4

code = [p1, p2, d1, p4, d2, d3, d4]
result += ''.join(map(str, code))

return result

text = input("Enter text: ")

binary = ''.join(format(ord(ch), '08b') for ch in text)

encoded = hamming_encode(binary)

with open("channel.txt", "w") as file:
    file.write(encoded)

print("Binary Data :", binary)
print("Hamming Code:", encoded)
print("Data saved in channel.txt")

```

Receiver Program

```

Python
def hamming_decode(code):
    data = ""

    for i in range(0, len(code), 7):
        block = list(map(int, code[i:i+7]))

```

```

if len(block) < 7:
    continue

# Calculate syndrome
s1 = block[0] ^ block[2] ^ block[4] ^ block[6]
s2 = block[1] ^ block[2] ^ block[5] ^ block[6]
s4 = block[3] ^ block[4] ^ block[5] ^ block[6]

error_position = s1 + (s2 * 2) + (s4 * 4)

if error_position != 0:
    print("Error detected at position:", error_position)
    block[error_position - 1] ^= 1
    print("Error corrected.")

# Remove parity bits
data += str(block[2])
data += str(block[4])
data += str(block[5])
data += str(block[6])

return data

with open("channel.txt", "r") as file:
    encoded = file.read().strip()

binary = hamming_decode(encoded)

text = ""

for i in range(0, len(binary), 8):
    byte = binary[i:i+8]

if len(byte) == 8:
    text += chr(int(byte, 2))

```

```
print("Received Data:", text)
```

TEST RUN

Input:

```
None  
HELLO
```

Sender Output:

```
None  
Binary Data : 0100100001000101010011000100110001001111  
Hamming Code: <encoded binary data>  
Data saved in channel.txt
```

Receiver Output:

```
None  
Received Data: HELLO
```

If one bit in `channel.txt` is changed manually, the receiver detects and corrects the error:

```
None  
Error detected at position: 5  
Error corrected.  
Received Data: HELLO
```

RESULT

Thus, the **Hamming Code** technique was successfully implemented to detect and correct a single-bit error in the transmitted data.